30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 16

Background Work — Part 2: Services, Foreground Services, Alarms & Push



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16

Why part 2 matters in 2026

Part 1 covered deferrable work. Part 2 is everything else: work the user is **watching**, work that must happen at a **certain time**, and work your **server** triggers. This is where Android changed the most — and where old answers crash on modern targetSdk:

- Services are still the building block — but a **background** service barely exists anymore
- Every foreground service needs a **type** and a matching permission (Android 14)
- Starting a foreground service **from the background** is mostly forbidden (Android 12)
- Exact alarms need a **permission** the user can deny
- Big user-initiated transfers get their own job type: **UIDT** (Android 14)

Level	Questions
Junior	Service types · Started services · Bound services
Mid	Foreground services · FGS types · Starting from the background · Alarms
Senior	JobScheduler & UIDT · Boot & rescheduling · FCM high priority
Architect	Choosing the right tool · Migrating to targetSdk 35+ · Reliable reminders



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: what keeps the work alive?

Mechanism	What keeps the work alive
viewModelScope	the screen
Foreground service	a visible notification
UIDT job	user action + notification
WorkManager / job	the system scheduler
Alarm	a moment in time
High-priority FCM	your server

- Every mechanism is a **contract** with the system — and each one has a price (a notification, a permission, a quota)
- A plain background service keeps nothing alive: the system stops it shortly after the app leaves the foreground
- Pick the **weakest** mechanism that still meets the requirement

THE ONE RULE BEHIND EVERY ANSWER TODAY

If neither the user nor the system scheduler knows about your work, assume it will be killed.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16



Junior

Service types · Started services · Bound services



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Services in 2026: three kinds

Kind	Lifetime	Reality today
Started — `startService()`	Until `stopSelf()` or `stopService()`	Can't be started from the background (Android 8+); stopped a few minutes after the app leaves the foreground
Bound — `bindService()`	Until the last client unbinds	Fine for in-app or cross-process APIs; not a way to stay alive
Foreground — `startForegroundService()`, then `startForeground()`	Until stopped; killed only under extreme memory pressure	Notification, type and permission required

`IntentService` (deprecated in API 30) and `JobIntentService` are gone from modern code — use WorkManager.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"A Service runs in the background, so it's off the main thread." — `onCreate()`, `onStartCommand()` and `onBind()` all run on the **main thread**. A Service is a component the system knows about, not a thread. Blocking it causes ANRs.

INTERVIEW TRAP

"A foreground service keeps running even when the app is killed." — Last year's version said this. The service lives **inside your process**: if the process dies, the service dies with it. Foreground status makes a kill much less likely — not impossible.

FOLLOW-UP

"Then why still write services?" — For work the user is actively aware of (playback, navigation, a call) and for exposing an API to other components or apps (bound services). Everything deferrable goes to WorkManager.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. A started service, done right

```
class ExportService : Service() {
    private val scope =
        CoroutineScope(SupervisorJob() + Dispatchers.Default)

    override fun onStartCommand(
        intent: Intent?, flags: Int, startId: Int,
    ): Int {
        scope.launch {
            exporter.export()
            stopSelf(startId)
        }
        return START_NOT_STICKY
    }

    override fun onDestroy() {
        scope.cancel()
        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Return values and stopping

`onStartCommand()` returns	If the process is killed
`START_NOT_STICKY`	Not recreated — the right default
`START_STICKY`	Recreated later, with a **null** intent
`START_REDELIVER_INTENT`	Recreated, and the last intent is delivered again

- `stopSelf(startId)` stops only if no newer start request arrived; `stopSelf()` stops regardless — and can kill work from a later `startService()`
- Cancel your scope in `onDestroy()`, or coroutines outlive the service

INTERVIEW TRAP

"*`START_STICKY` keeps my service alive.*" — It's a restart hint, not a keep-alive: the restart can be delayed, the intent is `null`, and every background limit still applies.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Bound services

```
class PlayerService : Service() {
    inner class LocalBinder : Binder() {
        fun service(): PlayerService = this@PlayerService
    }
    private val binder = LocalBinder()
    override fun onBind(intent: Intent): IBinder = binder
}

// Client: bind in onStart(), unbind in onStop()
val connection = object : ServiceConnection {
    override fun onServiceConnected(
        name: ComponentName, binder: IBinder,
    ) {
        player = (binder as PlayerService.LocalBinder).service()
    }
    override fun onServiceDisconnected(name: ComponentName) {
        player = null
    }
}
```

- `bindService(intent, connection, Context.BIND_AUTO_CREATE)` creates the service if needed; it's destroyed when the last client unbinds (unless it was also started)



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The binder details that matter

- A local `Binder` works only **in the same process**. Across processes: `Messenger` (one queue) or **AIDL** (concurrent calls)
- AIDL calls from another process arrive on a **binder thread pool**, not the main thread — the implementation must be thread-safe
- Media apps: Media3 `MediaSessionService` handles binding and the foreground service for you

INTERVIEW TRAP

"*`onServiceDisconnected()` is called when I unbind.*" — It isn't. It's called only when the connection is **lost unexpectedly** (the service's process crashed or was killed). Clear your reference yourself when you call `unbindService()`.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16

Mid-Level

Foreground services · FGS types · Starting from the background · Alarms



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. A foreground service in 2026

```
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />

<service
    android:name=".tracking.RunTrackingService"
    android:foregroundServiceType="location"
    android:exported="false" />
```

```
class RunTrackingService : LifecycleService() {
    override fun onStartCommand(
        intent: Intent?, flags: Int, startId: Int,
    ): Int {
        super.onStartCommand(intent, flags, startId)
        ServiceCompat.startForeground(
            this, NOTIFICATION_ID, trackingNotification(),
            ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION,
        )
        lifecycleScope.launch { tracker.track() }
        return START_NOT_STICKY
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The checklist

1. `FOREGROUND_SERVICE` permission (Android 9+) **and** the type's permission, e.g. `FOREGROUND_SERVICE_LOCATION` (Android 14+)
2. `android:foregroundServiceType` in the manifest, and the same type passed to `startForeground()`
3. A notification on a channel. Without `POST_NOTIFICATIONS` (Android 13+) the service still runs, but its notification shows only in the Task Manager
4. After `startForegroundService()`, call `startForeground()` within the documented few-second window — otherwise the app ANRs or crashes
5. Type prerequisites are checked at start: `location` needs a granted location permission, or `startForeground()` throws `SecurityException`
6. Stop with `ServiceCompat.stopForeground(this, STOP_FOREGROUND_REMOVE)` and `stopSelf()` — `stopForeground(true)` is deprecated

INTERVIEW TRAP

"Declare `FOREGROUND_SERVICE_CAPABILITY`; types are mandatory since Android 12." — Both wrong (last year's version). That permission **doesn't exist**. Types became mandatory for apps targeting **Android 14**; before that only location (10) and camera/microphone (11) access required one.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Choosing the FGS type

Type	Typical use	Also needs
`mediaPlayback`	Audio/video continuing in the background	`FOREGROUND_SERVICE_MEDIA_PLAYBACK`
`location`	Navigation, run tracking	Its permission + a granted location permission
`connectedDevice`	Wearables, Bluetooth accessories	Its permission + e.g. a Bluetooth permission
`phoneCall`	An ongoing call	Its permission + a call role or permission
`dataSync`	Upload, download, backup	`FOREGROUND_SERVICE_DATA_SYNC` — time-limited (15)
`shortService`	Finish a short critical task	No type permission; about 3 minutes, then `onTimeout()`
`specialUse`	Nothing else fits	Its permission + a manifest `<property>` explaining the use

- Apps targeting Android 14+ also **declare their FGS types in Play Console**, with a justification — a wrong type is a review rejection



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Android 15 time limits

ANDROID 15 (API 35)

For apps targeting API 35, `dataSync` (and the new `mediaProcessing` type) may run **6 hours in a 24-hour period**. At the limit the system calls `Service.onTimeout(startId, fgsType)`; stop within a few seconds or the app crashes. The timer resets when the user brings the app to the foreground.

```
override fun onTimeout(startId: Int, fgsType: Int) {
    checkpointStore.save(currentProgress)
    stopSelf(startId)
}
```

INTERVIEW TRAP

"*`dataSync` is the default type for anything in the background.*" — That's how apps got flagged. Deferrable sync → WorkManager. A big transfer the user started → a UIDT job (Q8). `dataSync` is for the cases in between.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Starting an FGS from the background

Since **Android 12** (targetSdk 31+), a background app can't start a foreground service — except in documented exemptions, including:

- The app was **just visible** (an activity was on screen moments ago)
- The user tapped your **notification, widget or bubble**
- An **exact alarm** fires for an action the user requested
- A **high-priority FCM** message arrives
- `BOOT_COMPLETED`, `LOCKED_BOOT_COMPLETED` or `MY_PACKAGE_REPLACED` (with Android 15 limits, Q9)
- The user turned **battery optimization off** for your app

```
try {
    ContextCompat.startForegroundService(context, intent)
} catch (e: IllegalStateException) {
    val notAllowed = Build.VERSION.SDK_INT >= 31 &&
        e is ForegroundServiceStartNotAllowedException
    if (notAllowed) scheduler.enqueueExpedited(task)
    else throw e
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Two traps in one rule

INTERVIEW TRAP

"I'm exempt, so my FGS can use the camera, microphone or location." — Starting is one check; **while-in-use access** is another (Android 11). An FGS started from the background can't use location, camera or microphone unless the app has the relevant access or a specific exemption applies.

INTERVIEW TRAP

"Send a high-priority push to start any FGS." — FCM **deprioritizes** high-priority messages that don't lead to a notification the user sees (Q10). A push-driven design that silently starts services breaks in production.

FOLLOW-UP

"What should the fallback be?" — Expedited WorkManager for short urgent work; otherwise show a notification that lets the user bring the app forward and start the FGS from there.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Alarms: inexact, exact, alarm clock

API	Precision	Exact-alarm permission (12+)
`set()`, `setWindow()`, `setInexactRepeating()`	Inexact, batched	Not needed
`setAndAllowWhileIdle()`	Inexact, fires in Doze (rate-limited)	Not needed
`setExact()`, `setExactAndAllowWhileIdle()`	Exact	**Required**
`setAlarmClock()`	Exact, shown to the user, wakes from Doze	**Required**

Permission	Since	For	Granted
`SCHEDULE_EXACT_ALARM`	Android 12	Apps with a real need	User can revoke; **denied by default** for most new installs on Android 14+
`USE_EXACT_ALARM`	Android 13	Alarm clock, timer, calendar apps — Play policy	At install; can't be revoked



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Scheduling an exact alarm safely

```
val canExact = Build.VERSION.SDK_INT < Build.VERSION_CODES.S ||
    alarmManager.canScheduleExactAlarms()

if (canExact) {
    alarmManager.setExactAndAllowWhileIdle(
        AlarmManager.RTC_WAKEUP, triggerAtMillis, pendingIntent,
    )
} else {
    alarmManager.setAndAllowWhileIdle(
        AlarmManager.RTC_WAKEUP, triggerAtMillis, pendingIntent,
    )
    // Offer Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM
}
```

- Calling an exact API without the permission throws `SecurityException`
- Listen for `ACTION_SCHEDULE_EXACT_ALARM_PERMISSION_STATE_CHANGED` to upgrade alarms when it's granted; if the user revokes it, the app is stopped and future exact alarms are cancelled

INTERVIEW TRAP

Exact alarms for periodic sync. They wake the device on your schedule, drain battery, and don't fit Play's exact-alarm policy. Sync belongs in WorkManager; alarms are for moments the **user** cares about.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16



Senior

JobScheduler & UIDT · Boot & rescheduling · FCM high priority



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. JobScheduler directly

```
class ReindexJobService : JobService() {
    private val scope = MainScope()

    override fun onStartJob(params: JobParameters): Boolean {
        scope.launch {
            indexer.reindex()
            jobFinished(params, false)
        }
        return true // work continues asynchronously
    }

    override fun onStopJob(params: JobParameters): Boolean {
        scope.coroutineContext.cancelChildren()
        return true // reschedule
    }
}
```

- Declare the service in the manifest with the `BIND_JOB_SERVICE` permission
- `setPersisted(true)` keeps the job across reboots (needs `RECEIVE_BOOT_COMPLETED`)
- WorkManager is the default; go direct for features WorkManager doesn't expose — like UIDT

INTERVIEW TRAP

"Using JobScheduler directly loses persistence." — Last year's version claimed this. `setPersisted(true)` survives reboots. What you lose is WorkManager's chaining, unique work, observability and test tooling.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. User-initiated data transfer (UIDT) jobs

```
val job = JobInfo.Builder(JOB_ID, component)
    .setUserInitiated(true)
    .setRequiredNetworkType(JobInfo.NETWORK_TYPE_ANY)
    .setEstimatedNetworkBytes(downloadBytes, 0L)
    .build()
jobScheduler.schedule(job)

// In the JobService
override fun onStartJob(params: JobParameters): Boolean {
    setNotification(
        params, NOTIFICATION_ID, progressNotification(),
        JobService.JOB_END_NOTIFICATION_POLICY_REMOVE,
    )
    startTransfer(params)
    return true
}
```

- Android 14 (API 34); permission `RUN_USER_INITIATED_JOBS`; must be **scheduled while the app is visible** (or in a few allowed states) and have a network constraint
- Can run for a long time when conditions allow; shows a notification the whole time
- Built for "download this season", "upload these 200 photos". Not exposed by WorkManager at the time of writing — use JobScheduler



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Boot and rescheduling

```
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        // Main thread, short budget: hand off to WorkManager
        val request = OneTimeWorkRequestBuilder<RescheduleWorker>()
            .build()
        WorkManager.getInstance(context).enqueueUniqueWork(
            "reschedule-reminders",
            ExistingWorkPolicy.KEEP,
            request,
        )
    }
}
```

- Manifest: `RECEIVE_BOOT_COMPLETED` + a receiver with a `BOOT_COMPLETED` intent filter (`exported="false"` works — the system still delivers it)

Scheduled with	After reboot	After force stop
WorkManager	Re-scheduled automatically	Re-scheduled on next app launch
JobScheduler + `setPersisted(true)`	Kept	Cancelled
AlarmManager	**Cleared** — re-schedule on boot	Cleared
Foreground service	Gone	Gone



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Boot traps

INTERVIEW TRAP

"My boot receiver runs right after install." — A newly installed app is in the **stopped state** until the user opens it once; it receives no broadcasts, including `BOOT_COMPLETED`. The same applies after a force stop.

INTERVIEW TRAP

Heavy work in `onReceive()`. It runs on the main thread with a short budget. Use `goAsync()` only for a few seconds of async work; anything longer → WorkManager.

ANDROID 15 (API 35)

For apps targeting API 35, a `BOOT_COMPLETED` receiver **can't start** foreground services of types `dataSync`, `camera`, `mediaPlayback`, `phoneCall`, `mediaProjection` or `microphone` — it throws `ForegroundServiceStartNotAllowedException`.

- **Direct Boot:** `BOOT_COMPLETED` arrives after the first unlock. Alarm clocks that must ring before it use `LOCKED_BOOT_COMPLETED`, `android:directBootAware` and device-protected storage — your normal Room database isn't available yet



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. FCM high-priority messages

```
class PushService : FirebaseMessagingService() {
    override fun onMessageReceived(message: RemoteMessage) {
        when (message.data["type"]) {
            "chat" -> notifier.showMessage(message.data)
            "sync" -> syncScheduler.requestSync()
        }
    }

    override fun onNewToken(token: String) {
        tokenRegistrar.register(token) // enqueues work
    }
}
```

	Normal priority	High priority
Device in Doze	Delivered at the next maintenance window	Delivered now; wakes the device
What the app may do	Handle it briefly	Short allowlist: network, expedited work, FGS start exemption
Use for	Sync hints, non-urgent updates	Something the user will **see**: a chat message, a call, an alarm



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. What breaks in production

INTERVIEW TRAP

High priority for silent syncs. If high-priority messages don't result in user-visible notifications, FCM can **deprioritize** them to normal. `RemoteMessage.priority` vs `originalPriority` tells you when that happened.

INTERVIEW TRAP

Long work in `onMessageReceived()`. The window is short — enqueue expedited WorkManager and return.

- **Notification messages** in the background go straight to the system tray; `onMessageReceived()` isn't called. Use **data messages** when the app must run logic
- Force-stopped and **hibernated** apps (unused for months) don't receive messages at all — not even high-priority ones

FOLLOW-UP

"Push or polling?" — Push to say "something changed", a periodic WorkManager sync as a safety net (Day 15, Q12).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16



Architect

Choosing the right tool · Migrating to targetSdk 35+ · Reliable reminders



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Choosing the right tool

```
Must it continue after the user leaves the screen?
├─ No  → coroutine in viewModelScope
└─ Yes → Must it happen at an exact time?
         ├─ Yes → exact alarm → notify or enqueue work
         └─ No  → Did the user start it and is watching?
                  ├─ Data transfer → UIDT job
                  ├─ Ongoing       → foreground service
                  └─ No            → WorkManager
                                     (expedited if urgent)
```

- Start at the top and stop at the **first** tool that satisfies the requirement — each step down costs more battery, permissions or review
- The server can start almost any branch: FCM → notification, expedited work or (exempt) FGS



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Scenarios interviewers love

Scenario	Tool
Refresh a list while the screen is open	Coroutine in `viewModelScope`
Send a chat message, even if the user leaves	Expedited WorkManager
Nightly backup on Wi-Fi and charging	Periodic WorkManager with constraints
Upload 200 photos the user just picked	UIDT on Android 14+; long-running worker below
Music playback	Media3 `MediaSessionService` (`mediaPlayback`)
Run tracking, turn-by-turn navigation	`location` foreground service
Wake-up alarm at 07:00	`setAlarmClock()` with `USE_EXACT_ALARM`
New order waiting on the server	High-priority FCM → notification

INTERVIEW TRAP

"WorkManager for everything" and *"a foreground service for everything"* are both wrong. The first fails user-visible work; the second fails Play review and drains battery.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Migrating a legacy app to targetSdk 35+

Area	What to do
Every FGS	Type + type permission in the manifest; type in `startForeground()`; Play Console declaration
`dataSync` services	Implement `onTimeout()`; move deferrable parts to WorkManager, user transfers to UIDT
Background starts	Find every `startForegroundService()` in receivers, pushes and workers; catch and fall back
Boot receivers	No `dataSync`, camera, microphone, media or phone-call FGS from `BOOT_COMPLETED`
Exact alarms	`canScheduleExactAlarms()` + inexact fallback; justify `USE_EXACT_ALARM` or drop it
Notifications	Request `POST_NOTIFICATIONS` at a moment that makes sense
Jobs	Test in the Restricted bucket and in Doze; expect stricter runtime quotas on Android 16

MY TAKE

Migrate in the order things **crash**: FGS types and background starts first (exceptions), then boot and alarm behavior (silent failures), then battery polish. Ship behind the targetSdk bump in a staged rollout.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Make regressions visible

- Track `ForegroundServiceStartNotAllowedException`, `MissingForegroundServiceTypeException` and `SecurityException` from `startForeground()` as **non-fatal** errors, tagged by Android version
- Test matrix on emulators: API 31, 34, 35, 36 — background start, reboot, exact alarm revoked, notifications denied
- Useful commands:

```
adb shell dumpsys activity services com.example.app
adb shell dumpsys alarm | grep com.example.app
adb shell am set-standby-bucket com.example.app restricted
```

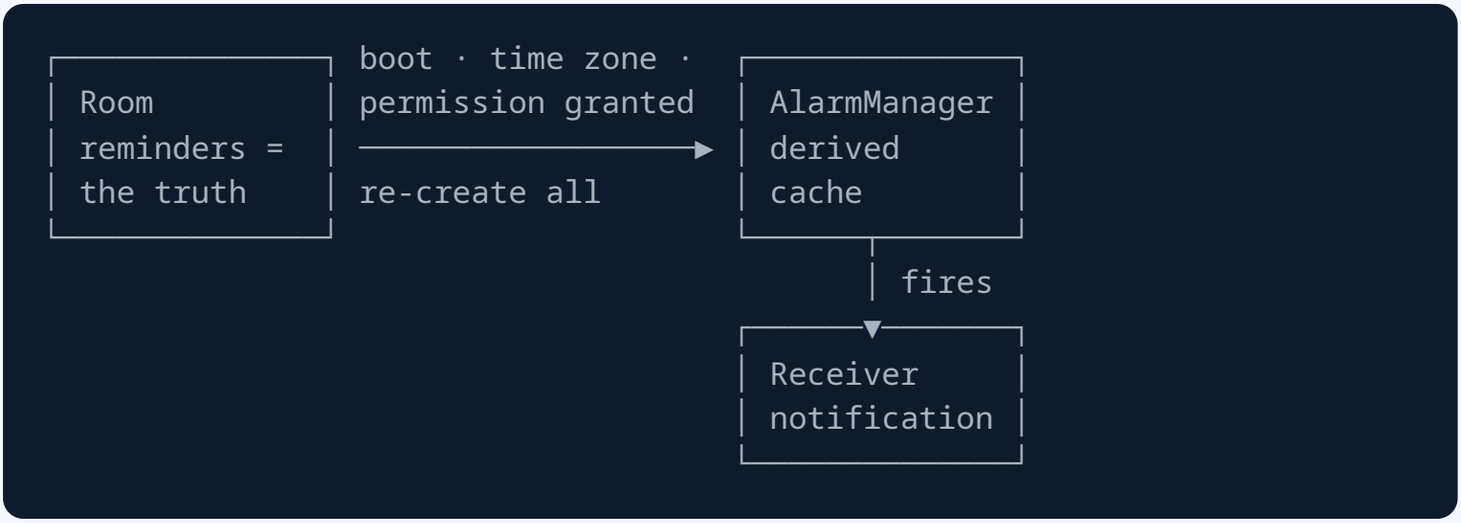
INTERVIEW TRAP

"Bump targetSdk, fix the build, ship." — Most of these changes don't break the build; they throw at runtime on specific versions, or silently stop firing. Without that test matrix, users find them first.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Design: reminders that actually fire



- Room is the **source of truth**; alarms are a derived cache that can be rebuilt at any time — idempotently
- Rebuild on: boot, `ACTION_TIMEZONE_CHANGED` / `ACTION_TIME_CHANGED` (both still allowed for manifest receivers), exact-alarm permission granted, and app start
- Exact alarm when allowed; otherwise inexact while-idle **and tell the user** reminders may be late, with a link to settings

INTERVIEW TRAP

"Schedule the alarm once, when the user creates the reminder." — A reboot or force stop wipes it, and a time-zone change shifts what "08:00" means. Without rebuilding from the source of truth, reminders silently stop.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. What makes it trustworthy

```
class ReminderReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val id = intent.getLongExtra(EXTRA_REMINDER_ID, -1)
        val title = intent.getStringExtra(EXTRA_TITLE)
        if (id < 0 || title == null) return
        val nm = NotificationManagerCompat.from(context)
        if (nm.areNotificationsEnabled()) {
            ReminderNotifier(context).show(id, title)
        }
        // Next occurrence: read Room off the main thread
        RescheduleWorker.enqueue(context, id)
    }
}
```

- `PendingIntent` with `FLAG_IMMUTABLE` and a stable request code per reminder, so re-scheduling **replaces** instead of duplicating
- Honest UX: if notifications are off, exact alarms denied or the app was force-stopped, **show it** in the app instead of failing silently

SENIOR ANSWER IN ONE LINE

"I pick the lightest contract with the system that meets the requirement — and I design for the moment the system says no: fallbacks, rescheduling from a source of truth, and honest UI."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 16 · ARCHITECT

Day 16 — cheat sheet

- Service callbacks run on the **main thread**; a foreground service dies with its process.
- Background services: no starting from the background (8.0), stopped soon after the app leaves.
- Bound services live while clients are bound; `onServiceDisconnected()` means a crash, not an unbind.
- Every FGS: notification + **type** + type permission (Android 14); `startForeground()` quickly.
- No FGS starts from the background (Android 12) outside the exemptions — catch and fall back.
- `dataSync` FGS: 6 h per 24 h (Android 15) — implement `onTimeout()`.
- Exact alarms need `SCHEDULE_EXACT_ALARM` (denied by default on 14+) or `USE_EXACT_ALARM` (alarm/calendar apps).
- UIDT jobs for big user-started transfers; JobScheduler `setPersisted(true)` survives reboot, alarms don't.
- High-priority FCM only for what the user sees — otherwise it's deprioritized.
- Architect: lightest mechanism first, a source of truth to rebuild from, and a fallback for every "no".

Day 17 is next → Performance, Part 1: rendering, jank, memory & profiling. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700